

A Course-Project Reproduction Study of GuP for Exact Subgraph Matching

YI ZHA, Tsinghua University, China

JINPENG WANG, Tsinghua University, China

This report presents a course-project reproduction study of GuP, an exact subgraph matching method based on guard-based pruning. We implemented a correctness-oriented baseline matcher and extended it with a *GUP-lite* design that supports reservation-based pruning, nogood-based pruning, metric collection, and filtering-strategy ablation. The study includes a literature taxonomy, a clear statement of reproduction scope, experiments on three successful datasets (Yeast, Human, and HPRD), and a time-boxed boundary evaluation on WordNet and Patents. Overall, reservation-based pruning is already effective on several real workloads and is the main source of measurable search-space reduction in our prototype, whereas the current nogood mechanism remains conservative and the Python implementation still shows a clear scalability boundary.

1 Introduction

Exact subgraph matching is a fundamental graph query problem with applications in social-network analysis, biological interaction analysis, fraud detection, and graph database systems. Given a large vertex-labeled data graph and a smaller query graph, the goal is to enumerate all injective mappings that preserve both labels and edges. Because the search space grows rapidly with query size, practical systems rely heavily on candidate filtering, matching-order optimization, and state-aware pruning.

In this project, we reproduce *GuP*, a recent exact subgraph matching method based on guard-based pruning [1]. GuP introduces two complementary pruning ideas: reservation guards, which propagate injectivity-related constraints to earlier search steps, and nogood guards, which attempt to reuse dead-end information discovered during the search. These mechanisms are designed to reduce futile recursive exploration while preserving exactness.

The goal of this course project is not only to obtain a runnable implementation, but also to present a clear and complete analysis. Accordingly, besides implementing and evaluating a correctness-oriented reproduction of GuP, we organize the report around four core components: literature review and taxonomy, reproduction of one selected paper, required experiments on multiple datasets with specified metrics, and ablation of filtering strategies.

1.1 Project Scope

Our implementation follows a staged strategy. We first build a baseline depth-first backtracking matcher, then add GuP-inspired reservation and nogood guards, and finally evaluate the resulting variants under a shared metric framework. The resulting system is still a course-project reproduction rather than a line-by-line reimplement of the authors' production code, but it already captures the main algorithmic components needed for comparative analysis.

1.2 Report Organization and Grading-Item Mapping

Table 1 summarizes how the report structure aligns with the required grading items.

The remainder of the report is organized as follows. Section 2 reviews the literature and builds a taxonomy for exact subgraph matching. Sections 3–5 define the problem, summarize GuP, and describe the reproduced system. Sections 6–8 present the required experiments and the ablation study. Section 9 concludes with a brief development-history discussion and personal reflection.

Authors' Contact Information: Yi Zha, Tsinghua University, Beijing, China, chay22@mails.tsinghua.edu.cn; Jinpeng Wang, Tsinghua University, Beijing, China, wjp21@mails.tsinghua.edu.cn.

Table 1. Mapping from the grading rubric to the report structure.

Grading item	Report sections	Main evidence
Literature review (15 pts)	Secs. 2; Tab. 2	Taxonomy and positioning of GuP
Paper reproduction (20 pts)	Secs. 4, 5; Tab. 3	Reproduced components and scope boundary
Required experiments (20 pts)	Secs. 6, 7; Tabs. 4, 6, 7	Runtime, result mappings, partial mappings, and paper-filtered partial mappings
Strategy ablation (15 pts)	Sec. 8; Tabs. 5, 8	Guard on/off comparison and contribution analysis

2 Literature Review and Multi-dimensional Taxonomy

Exact subgraph matching has been studied for more than one research generation, and representative progress has appeared from both the database and graph-algorithm communities. Prior work does not advance along a single axis: some methods emphasize aggressive candidate-space reduction before backtracking begins, some optimize the search order to expose contradictions earlier, some reuse dead-end information to avoid repeating equivalent failures, and others redesign the whole execution pipeline around candidate representation and set operations [1–6]. Therefore, a meaningful literature review for this course project should not merely list papers chronologically. Instead, it should provide a *multi-dimensional taxonomy* that explains how different methods trade off filtering power, search overhead, and implementation complexity, and how the reproduced GuP method fits into this larger design space.

2.1 Overview of Prior Work

Prior work on exact subgraph matching is highly diverse, so in this report we summarize it with a compact multi-dimensional taxonomy instead of a purely chronological list. The goal is to highlight the dominant optimization ideas behind representative methods and to clarify how GuP relates to them.

2.2 Taxonomy Dimensions

We classify representative work using four main dimensions.

- **Candidate filtering:** how aggressively a method shrinks the domain of each query vertex before or during the search. Typical signals include labels, degrees, neighborhood signatures, and local structural consistency. Stronger filtering reduces the branching factor, but it may require heavier preprocessing or more complex auxiliary data structures [3, 5].
- **Matching order:** how the method chooses the next query vertex to expand. Since exact matching is usually realized as state-space search, the order of expansion strongly affects whether contradictions appear early or late. Static heuristics are simple and cheap, while adaptive order selection can better react to the current partial embedding but also introduces additional control overhead [2, 5].
- **Conflict reuse / failing sets:** whether the method memorizes information learned from dead ends and reuses it to skip equivalent or dominated branches later. This dimension is especially important when many branches fail for similar reasons; in such cases, remembering the “shape” of failure may be more valuable than further tightening local consistency checks [1, 2].
- **System-level acceleration:** whether the method relies on heavier indexing, candidate-space engineering, specialized set operations, or a holistic execution framework beyond local pruning logic. This dimension separates algorithms that are conceptually lightweight but search-aware from methods whose performance depends heavily on broader system design [3, 6].

Table 2. A multi-dimensional taxonomy of representative exact subgraph matching studies.

Study / Method	Cand. filt.	Order	Conflict reuse	System support	Main emphasis	Relation to GuP
GraphQL [3]	Strong	Moderate	No	Yes	Signature-based pruning and graph access methods for reducing candidate space early	Earlier candidate-space-oriented line
DAF [2]	Strong	Strong	Strong	Moderate	Dynamic programming, adaptive matching order, and failing sets in one exact matcher	Closest classical conflict-aware reference
In-depth study [5]	Covers many	Covers many	Covers many	Covers many	Comparative analysis showing which components dominate performance across workloads	Evaluation vocabulary and taxonomy support
RapidMatch [6]	Strong	Strong	Limited	Strong	Holistic high-performance query processing instead of a single pruning trick	Stronger system emphasis than GuP
VEQ [4]	Moderate	Moderate	Limited	Moderate	Equivalence-aware reduction of redundant matching work	Alternative redundancy-reduction line
GuP [1]	Moderate	Moderate	Strong	Moderate	Guard-based pruning through reservation and nogood mechanisms	Our reproduction target

These dimensions are complementary rather than mutually exclusive. A strong exact matcher often combines several of them, but different papers prioritize different components. Accordingly, our taxonomy is intended to show *dominant emphasis* rather than force each method into a single exclusive box.

2.3 Representative Research Lines

Candidate-space-oriented filtering. An influential early database line focuses on reducing the candidate space as much as possible before expensive enumeration begins. GraphQL is a representative example: it integrates signature-based pruning and graph access methods so that many impossible matches can be removed early, before full backtracking search is triggered [3]. This line established an important intuition for later work: exact matching performance is often determined by how small and how accurate the candidate domains are. In practice, however, such methods can still struggle when local signatures are insufficiently discriminative, because substantial ambiguity remains and must be resolved by downstream search.

Integrated search optimization. A second line treats exact subgraph matching explicitly as a search problem whose major components should be optimized together. DAF is representative of this direction because it combines dynamic programming, adaptive matching order, and failing sets within one exact matcher [2]. The importance of DAF in our review is twofold. First, it shows that search order is not a secondary heuristic but a first-class performance factor: different orderings can expose conflicts at very different depths. Second, it demonstrates that failure information can be summarized and reused, rather than discarded after each dead end. Compared with earlier candidate-space-oriented methods, this line moves more of the optimization effort from static preprocessing to dynamic search control.

Comparative understanding of in-memory systems. The field has also developed a more reflective line of work that compares existing exact matchers systematically instead of proposing only one new algorithm. The in-depth study by Sun and Luo is especially valuable in this regard [5]. Rather than attributing performance to a single pruning idea, it analyzes how candidate generation, matching order, intermediate representation, and implementation decisions interact across workloads. For our course project, this study is useful because it provides a vocabulary for discussing why a method is fast or slow. It also supports our decision to report not only runtime, but also search-space indicators such as the number of partial mappings.

Holistic execution and system support. Another research line pushes exact matching toward a more holistic execution-engine perspective. RapidMatch is representative here: instead of emphasizing only one local pruning rule, it redesigns subgraph query processing around the coordinated treatment of candidate generation, query execution, and intermediate-state handling [6]. This line is important because it reminds us that state-of-the-art performance may rely not only on better pruning logic, but also on better organization of the entire execution pipeline. From a reproduction perspective, however, such methods are often harder to emulate faithfully in a course project, since low-level engineering choices contribute substantially to the final speedup.

Redundancy and equivalence reduction. A closely related but conceptually distinct line seeks to reduce redundant search by recognizing when different branches correspond to equivalent or near-equivalent computational states. VEQ is a representative method in this category [4]. Its contribution is useful for our taxonomy because it shows that repeated work can be reduced not only by stronger filtering or better failure memorization, but also by detecting equivalence relationships among search states. Compared with DAF-style failing sets, this direction is less about recording explicit dead ends and more about compressing duplicated work more broadly.

Relation among these lines. Taken together, these studies suggest that exact subgraph matching has evolved along several complementary directions rather than a single dominant path. Candidate-space methods try to shrink the search problem early; search-centric methods optimize how the search tree is traversed; conflict-reuse methods try to avoid rediscovering the same failures; and system-oriented methods improve the execution pipeline as a whole. This perspective is more useful for our report than a simple chronological list, because it directly supports the later analysis of GuP’s reservation guard and nogood guard.

2.4 Positioning of GuP in the Taxonomy

Under the above taxonomy, GuP is best understood as a *search-based exact matcher with dynamic state-aware pruning*. Its core contribution is not the heaviest pre-search filtering, nor the most system-heavy execution engine, but the introduction of guards that evaluate whether a candidate extension is likely to make the remaining search infeasible [1]. In particular, the reservation guard propagates future injectivity pressure earlier in the search, while the nogood guard attempts to reuse information from previously discovered dead ends. Therefore, GuP lies between classical candidate-space methods and stronger failing-set-style conflict-reuse methods: it still operates inside a backtracking enumeration framework, but it pushes pruning decisions closer to the dynamic search state.

This position makes GuP especially suitable for a course-project reproduction. Compared with GraphQL, GuP is less dependent on heavyweight offline candidate-space summaries; compared with DAF, it exposes its pruning ideas through relatively modular guards; compared with RapidMatch, it is less tied to full-stack system engineering; and compared with VEQ, its pruning contribution is easier to align with the course-required statistics on partial mappings and filtered partial mappings [1–4, 6]. In other words, GuP offers a favorable balance between research novelty, conceptual clarity, ablation friendliness, and implementability.

2.5 Implications for the Rest of This Report

The literature review leads to three concrete decisions for the rest of our report. First, we treat GuP as part of the broader conflict-aware exact matching line rather than as an isolated technique. Second, we evaluate our prototype not only by end-to-end runtime but also by search-space indicators that reflect how the guards affect enumeration. Third, when discussing limitations, we distinguish algorithmic limitations from system-level ones: if our Python reproduction

underperforms highly optimized prior systems, this does not automatically invalidate the pruning ideas themselves. This distinction is important for interpreting experimental results fairly in a course-project setting.

3 Problem Definition and Evaluation Metrics

Let the data graph be $G = (V_G, E_G, \ell_G)$ and the query graph be $Q = (V_Q, E_Q, \ell_Q)$, where vertices are associated with discrete labels. The exact subgraph matching problem asks for all injective mappings

$$f : V_Q \rightarrow V_G$$

such that:

- (1) label preservation holds, that is, $\ell_Q(u) = \ell_G(f(u))$ for every query vertex $u \in V_Q$, and
- (2) edge preservation holds, that is, $(u, u') \in E_Q$ implies $(f(u), f(u')) \in E_G$.

In our implementation, all graphs are undirected, vertex-labeled, and loop-free. We do not consider edge labels, subgraph homomorphism, or dynamic graph updates in the mandatory part of the project.

3.1 Required Metrics

The project specification requires four core metrics. We make the counting rules explicit so that all experimental variants are directly comparable.

- **Runtime (ms):** wall-clock runtime of the matching procedure, excluding file-loading overhead.
- **Result mappings:** the number of complete valid embeddings returned by the matcher.
- **Partial mappings:** the number of successful partial extensions generated during the search.
- **Paper-filtered partial mappings:** the number of partial extensions pruned specifically by the reproduced paper's filtering strategies.

The last metric requires special clarification. Our internal counter `pruned_partial_mappings` records *all* rejected extensions, including injectivity and edge-consistency conflicts. However, the assignment specifically asks for the number of partial mappings filtered out by the paper's own strategies. Therefore, in the report we separately extract:

- `reservation_guard_filtered`,
- `nogood_guard_filtered`, and
- `paper_filter_filtered_total` = `reservation` + `nogood`.

This distinction is important because the total prune count reflects the entire feasibility-check framework, whereas the paper-filtered count isolates the contribution of the reproduced GuP-style filtering strategies.

4 Reproduced Paper and Reproduction Scope

The selected paper is *GuP: Fast Subgraph Matching by Guard-Based Pruning* [1]. The central insight of GuP is that many dead-end partial embeddings can be rejected before an explicit structural violation is reached. To achieve this, GuP augments the search with guard-based pruning.

4.1 Main Idea of GuP

The first mechanism is the *reservation guard*. Intuitively, when a candidate assignment (u, v) is made, some future query vertices may already be forced to consume a small set of data vertices. If one of those data vertices has already been used too early, the extension can be rejected immediately.

Table 3. Reproduction scope of the course-project GuP implementation.

Original GuP component	Implemented	Our design	Faithfulness	Expected impact on results
Baseline exact search	Yes	Depth-first backtracking matcher	High	Shared correctness reference
Candidate generation	Yes	Label + degree filtering	Partial	Weaker than richer candidate-space engineering
Matching order	Yes	Simple order by candidate size, degree, and id	Partial	May reduce pruning strength on harder workloads
Reservation guard	Yes	Precomputed small reservation sets with Hall-style reasoning	Medium	Main source of search-space reduction in our results
Nogood guard	Yes	Conservative future-subproblem memoization	Partial	Safe but weaker than the original full mechanism
Guarded candidate-space engineering	Partial	Only the minimum structures required by the prototype	Partial	Limits peak performance and scalability
Low-level system optimization	No	Python correctness-first prototype	Low	Strongly affects wall-clock competitiveness on heavy workloads

The second mechanism is the *nogood guard*. Instead of relying only on local consistency checks, the search records signatures of dead-end situations and uses them to reject future extensions that induce the same remaining subproblem. In the original paper, this mechanism is tightly integrated with the guarded candidate space and richer guard discovery rules.

4.2 Reproduction Scope

Table 3 summarizes which original GuP components were reproduced in this project.

The most important boundary is therefore not correctness but *faithfulness of engineering detail*. Our reproduction is sufficient for analyzing the main algorithmic ideas and for ablating the two guard mechanisms, but it should not be interpreted as a claim of matching the original production-level performance of GuP on every workload.

5 Implementation Details

5.1 Implementation Overview

We implemented the project in Python with a correctness-oriented design philosophy. The codebase contains two main matchers: a baseline depth-first backtracking matcher and a *GUP-lite* matcher that reuses the same search framework but adds guard-based pruning. This shared architecture is important for the grading-oriented comparison because it allows the effect of individual filtering strategies to be isolated cleanly in Section 8.

5.2 Graph Representation and Input Handling

All graph variants are converted into a unified in-memory structure `LabeledGraph`, which stores an adjacency dictionary for undirected edges and a label dictionary for vertex labels. The project currently supports three input formats: a toy text format (`v/e`), the official GuP example format (`.vertices`, `.edges`, and YAML query sets), and the `.graph` format used by the downloaded benchmark package.

5.3 Baseline Matcher

The baseline matcher constructs candidate sets, computes a matching order, and performs recursive backtracking. Each extension is validated by injectivity and edge-consistency checks. The baseline already records `result_mappings`, `partial_mappings`, `pruned_partial_mappings`, and `runtime_ms`, which makes it the common reference point for both the required multi-dataset experiments and the filtering-strategy ablation.

5.4 Candidate Generation and Matching Order

Candidate generation was upgraded from pure label filtering to a `label + degree` rule: a data vertex is kept as a candidate only if the labels match and the data-graph degree is at least the query-graph degree. The matching order is determined by ascending candidate-set size, descending degree, and vertex identifier as a tiebreaker. This part corresponds to the reproduced paper’s candidate-space preparation, although it remains simpler than the richer engineering used in the original GuP implementation.

5.5 Reservation Guard

The reservation guard evolved through two stages. The earlier version behaved as a lightweight runtime forward check. The improved version precomputes small reservation sets before search. For a candidate assignment (u, v) , it inspects forward neighbors of u in the matching order and collects compatible future candidates. If a small subset of those future candidates satisfies a Hall-style equality between subset size and the size of the candidate union, the union is treated as reserved. At runtime, the guard only tests whether one of those reserved data vertices has already been used.

This component corresponds directly to one of the filtering strategies studied in the report. In the final experiments, it is also the main source of nonzero paper-filtered partial mappings.

5.6 Nogood Guard

The current nogood guard is a conservative memoization mechanism rather than a full reproduction of the original GuP nogood machinery. Instead of implementing the full search-node encoding from the paper, the current version stores a signature of the future subproblem after a candidate extension, including the search depth, the hypothetical used data vertices, and the frontier mapping. Later in the project, we further reduced overhead by activating nogood checks only on query vertices that had already accumulated relevant dead-end information.

This component corresponds to the second filtering strategy in the required ablation. In our reproduction it remains safe but conservative, which explains why its explicit pruning contribution is limited on the real datasets.

5.7 Metrics and Experiment Scripts

All variants share the same metric object, which records counts for results, partial mappings, pruned extensions, guard checks, guard-specific prune reasons, and runtime. To support reproducible experiments, we built a small workflow around three layers: single-run scripts, batch scripts for standard ablation, and result-summarization scripts that aggregate JSON outputs into report-oriented CSV tables.

5.8 Current Limitations

The implementation still differs from the original paper in several ways: the nogood mechanism remains simplified, the guarded candidate space is not fully reconstructed, the matching order is more basic than in GuP, and the entire

Table 4. Datasets used in the report.

Dataset	$ V $	$ E $	Queries used	Role
Yeast	3112	12519	dense-4: 1, 10	Main
Human	4674	86282	dense-4: 1, 10	Main
HPRD	9460	34998	dense-4: 1, 10	Main
WordNet	76853	120399	dense-4: 1; sparse-8: 15	Boundary
Patents	3774768	16518947	dense-4: 1; sparse-8: 15	Boundary

Table 5. Filtering-strategy switches used in the experiments.

Variant	Reservation guard	Nogood guard
Baseline	Off	Off
Reservation only	On	Off
Nogood only	Off	On
Full GUP	On	On

system is implemented in Python. Nevertheless, the implementation is sufficient to support the main analyses in this report: one-paper reproduction, multiple-dataset evaluation with explicit metrics, and direct ablation of the reproduced filtering strategies.

6 Experimental Setup

The benchmark data were taken from the public dataset package used in the GuP study and loaded through the project’s unified .graph parser. For clarity, we explicitly separate *main successful datasets* from *time-boxed boundary datasets*.

The final mandatory evaluation is based on three successful datasets: Yeast, Human, and HPRD. For each of these datasets, we report results on two real benchmark queries so that the report covers both lighter and heavier search regimes. WordNet and Patents are retained as time-boxed boundary datasets because the current Python prototype still reaches its scalability limit on them.

6.1 Compared Variants

We compare four variants, corresponding directly to the required filtering-strategy ablation.

6.2 Measured Metrics and Time-box Policy

For every successful run, we record runtime, result mappings, partial mappings, and paper-filtered partial mappings. The paper-filtered count is computed from the prune-reason log and currently consists of the number of extensions rejected by the reservation and nogood guards. Because the current implementation rarely activates useful nogood pruning on the main workloads, the paper-filtered count is dominated by the reservation guard in the final results.

For boundary datasets, we use a 120-second time box. If a run does not finish within this limit, it is explicitly reported as timeout instead of being silently excluded. This makes the scalability boundary of the prototype explicit and reproducible.

7 Required Experimental Results on Multiple Datasets

Table 6 reports the mandatory evaluation results on the three successful datasets. The table lists the four required metrics: runtime, result mappings, partial mappings, and the number of partial mappings filtered by the reproduced paper’s strategies.

Table 6. Required metrics on the three successful datasets. ‘Paper-filtered PM’ denotes the number of partial mappings pruned by GuP-style filtering strategies only.

Dataset	Query	Variant	Result mappings	Partial mappings	Paper-filtered PM	Runtime (ms)
HPRD	d4-1	Baseline	980	1514	0	237.90
HPRD	d4-1	Full GUP	980	1440	74	208.63
HPRD	d4-1	Nogood only	980	1514	0	224.05
HPRD	d4-1	Reservation only	980	1440	74	211.05
HPRD	d4-10	Baseline	4	51	0	8.96
HPRD	d4-10	Full GUP	4	11	27	1.35
HPRD	d4-10	Nogood only	4	51	0	6.88
HPRD	d4-10	Reservation only	4	11	27	1.36
Human	d4-1	Baseline	10312	11013	0	185.86
Human	d4-1	Full GUP	10312	10916	97	143.34
Human	d4-1	Nogood only	10312	11013	0	161.53
Human	d4-1	Reservation only	10312	10916	97	170.25
Human	d4-10	Baseline	12830306	12937898	0	50314.48
Human	d4-10	Full GUP	12830306	12937608	290	56733.61
Human	d4-10	Nogood only	12830306	12937898	0	55239.79
Human	d4-10	Reservation only	12830306	12937608	290	55887.18
Yeast	d4-1	Baseline	720	772	0	19.27
Yeast	d4-1	Full GUP	720	772	0	27.72
Yeast	d4-1	Nogood only	720	772	0	22.68
Yeast	d4-1	Reservation only	720	772	0	19.11
Yeast	d4-10	Baseline	19678	22624	0	805.98
Yeast	d4-10	Full GUP	19678	22283	341	731.28
Yeast	d4-10	Nogood only	19678	22624	0	809.12
Yeast	d4-10	Reservation only	19678	22283	341	682.42

Taken together, the three successful datasets support three main observations. First, the report now satisfies the “3–4 datasets” requirement with Yeast, Human, and HPRD as complete successful datasets.

Second, among the reproduced paper strategies, only the reservation guard contributes nonzero paper-filtered partial mappings consistently on the real workloads considered here.

Table 7. Time-boxed boundary datasets under the 120-second protocol.

Dataset	Query	Time limit (s)	Status
WordNet	d4-1	120	timeout
WordNet	s8-15	120	timeout
Patents	d4-1	120	timeout
Patents	s8-15	120	timeout

Third, the benefit of reservation-based pruning is clearly workload-dependent: it is strongest on HPRD d4-10, moderate on HPRD d4-1 and Yeast d4-10, and very small on Human d4-10.

HPRD is especially informative because it fills the gap between the smaller datasets and the boundary workloads. On HPRD d4-10, reservation-based pruning reduces the number of partial mappings from 51 to 11 and improves runtime from 8.96 ms to about 1.35 ms. On HPRD d4-1, it reduces the search space by 74 partial mappings and improves runtime by about 12%.

The results on Human and Yeast are more mixed, but still informative. On Human d4-1, full GUP achieves the best runtime among all tested variants. On Yeast d4-10, both reservation-only and full GUP improve runtime, and reservation-only gives the strongest speedup. By contrast, on Human d4-10, the reservation guard still reduces the search space slightly, but the prototype does not convert that gain into lower runtime.

Table 7 records the boundary workloads explicitly rather than omitting them. These runs do not contribute complete metric rows, but they remain important because they mark the scalability range of the Python prototype. The report therefore includes both the required successful datasets and a transparent account of the workloads that remain beyond the current implementation boundary.

8 Ablation Study of Filtering Strategies

To assess the contribution of each filtering strategy, we disable them in turn and compare the resulting changes in runtime and partial mappings. Table 8 reports this comparison directly.

Table 8. Ablation results relative to the baseline. Negative deltas indicate improvement over the baseline.

Dataset	Query	Variant	Partial delta	Partial delta (%)	Runtime delta (ms)
HPRD	d4-1	Reservation only	-74	-4.89	-26.85
HPRD	d4-1	Nogood only	0	0.00	-13.85
HPRD	d4-1	Full GUP	-74	-4.89	-29.27
HPRD	d4-10	Reservation only	-40	-78.43	-7.60
HPRD	d4-10	Nogood only	0	0.00	-2.08
HPRD	d4-10	Full GUP	-40	-78.43	-7.61
Human	d4-1	Reservation only	-97	-0.88	-15.61
Human	d4-1	Nogood only	0	0.00	-24.33
Human	d4-1	Full GUP	-97	-0.88	-42.52
Human	d4-10	Reservation only	-290	-0.00	5572.70
Human	d4-10	Nogood only	0	0.00	4925.30

Dataset	Query	Variant	Partial delta	Partial delta (%)	Runtime delta (ms)
Human	d4-10	Full GUP	-290	-0.00	6419.13
Yeast	d4-1	Reservation only	0	0.00	-0.15
Yeast	d4-1	Nogood only	0	0.00	3.42
Yeast	d4-1	Full GUP	0	0.00	8.45
Yeast	d4-10	Reservation only	-341	-1.51	-123.56
Yeast	d4-10	Nogood only	0	0.00	3.14
Yeast	d4-10	Full GUP	-341	-1.51	-74.71

The ablation results support three concise conclusions.

First, the **reservation guard is the dominant effective filtering strategy** in our reproduction. It is responsible for every nonzero paper-filtered partial-mapping count in the main experiments and for the strongest speedups, especially on HPRD d4-10 and Yeast d4-10.

Second, the **nogood guard remains conservative in pruning power**. In the present implementation it rarely contributes explicit nogood-based pruning on the successful real workloads, which is why its partial-mapping delta is usually zero. Even so, after reducing unnecessary guard checks, the nogood-only variant is not always slower than the baseline; on some lighter workloads it is mildly beneficial in runtime even without visible search-space reduction.

Third, the **combination of both guards is workload-dependent**. On HPRD and on Human d4-1, full GUP inherits the benefit of reservation-based pruning and outperforms the baseline. On the heavier Human d4-10, however, the prototype still pays more guard overhead than it saves. Overall, the ablation study shows that reservation-based pruning already contributes measurable value, whereas nogood-based pruning still requires a more faithful implementation to demonstrate stable gains.

9 Conclusion, Development Context, and Reflection

This project reproduced the core idea of GuP through a correctness-oriented implementation for exact subgraph matching. The final report includes a literature taxonomy, a clear reproduction-scope statement, mandatory experiments on three successful datasets, and a filtering-strategy ablation study.

From the perspective of the development history of subgraph matching, the field has evolved from earlier candidate-space and access-method ideas [3] toward increasingly integrated systems that combine filtering, order optimization, conflict reuse, and execution engineering [2, 5, 6]. GuP is an instructive modern point in this development line because it emphasizes dynamic, state-aware pruning while remaining conceptually decomposable into interpretable modules.

Our results lead to three main conclusions. First, the project now satisfies the mandatory multi-dataset requirement with successful experiments on Yeast, Human, and HPRD. Second, reservation-based pruning is the only reproduced GuP mechanism that contributes clear and repeated search-space reductions on the current workloads. Third, the current Python prototype still has a visible scalability boundary, which is why WordNet and Patents remain time-boxed boundary datasets rather than full successful result datasets.

The most important lesson for us is that reproducing a research paper is not only about matching the final metric values. It also requires understanding which algorithmic ideas are central, which engineering details control practical performance, and how to report scope boundaries honestly. In this project, the largest gap between the paper and the

prototype lies not in correctness, but in the strength of the full guarded candidate-space machinery and the efficiency of the nogood mechanism. This gave us a concrete understanding of the difference between an algorithmic idea that is valid in principle and a system implementation that is competitive in practice.

Overall, we regard the project as a successful course-project-level study of the core GuP method. It shows that guard-based pruning, especially reservation-based pruning, can reduce the search space on real workloads, while also clarifying what would need to be improved in a more faithful future reproduction.

10 Division of Labor

The project was completed by a two-person team.

- **Yi Zha:** framework construction, algorithm reproduction, end-to-end pipeline integration, and report writing.
- **Jinpeng Wang:** algorithm optimization, experimental execution, and presentation slide preparation.

Both members jointly participated in the final presentation and oral explanation of the project.

References

- [1] Junya Arai, Yasuhiro Fujiwara, and Makoto Onizuka. 2023. GuP: Fast Subgraph Matching by Guard-Based Pruning. *Proceedings of the ACM on Management of Data* 1, 2, Article 167 (June 2023), 26 pages. doi:10.1145/3589312
- [2] Minseok Han, Hwanjo Kim, Gyeong Gu, Kunsoo Park, and Wook-Shin Han. 2019. Efficient Subgraph Matching: Harmonizing Dynamic Programming, Adaptive Matching Order, and Failing Set Together. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD '19)*. Association for Computing Machinery, New York, NY, USA, 1429–1446. doi:10.1145/3299869.3319880
- [3] Huahai He and Ambuj K. Singh. 2008. Graphs-at-a-Time: Query Language and Access Methods for Graph Databases. In *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data (SIGMOD '08)*. Association for Computing Machinery, New York, NY, USA, 405–418. doi:10.1145/1376616.1376655
- [4] Hyunjoon Kim, Yunyoung Choi, Kunsoo Park, Xuemin Lin, Seok-Hee Hong, and Wook-Shin Han. 2021. Versatile Equivalences: Speeding up Subgraph Query Processing and Subgraph Matching. In *Proceedings of the 2021 International Conference on Management of Data (SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 925–937. doi:10.1145/3448016.3457265
- [5] Shixuan Sun and Qiong Luo. 2020. In-Memory Subgraph Matching: An In-Depth Study. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 1083–1098. doi:10.1145/3318464.3380581
- [6] Shixuan Sun, Xibo Sun, Yulin Che, Qiong Luo, and Bingsheng He. 2020. RapidMatch: A Holistic Approach to Subgraph Query Processing. *Proceedings of the VLDB Endowment* 14, 2 (2020), 176–188. doi:10.14778/3425879.3425888